【扔物线 Compose 课程讲义】4.2-流程定制型动画 Animatable 和 LaunchedEffect()

关于讲义

讲义是对课程内容中**所有结论的集中汇总**。所以在看完每节课程之后，迅速来看一遍讲义，对课程内容做回顾，会有非常好的吸收效果。

面试前不要背讲义

在面试之前，通过快速浏览一遍讲义来确认所有知识都没有忘记，是个很好的备战方式。

但**不要用【背讲义上的知识点】的方式来备战**。最有效的浏览方式是：

浏览讲义中的所有内容，并：

- 看到「懂，并且记得背后的原因（或技术细节）」的，直接跳过；
- 看到「懂，但细节已经忘了」的，快速回看当节课程，把知识捡起来；
- 看到「已经不太懂 / 完全忘了」的，认真重看当节课程，把知识学回来。

疑问或意见

如果你发现讲义或课程有错误，或有别的什么意见，可以向任何一个你认识的助教反馈。

如果认可我们的课程，还请给个好评！ 评价链接：https://rengwuxian.com/cp

为什么用 `Animatable`？

因为它可以实现动画流程细节的定制，虽然写起来没有 `animateXxxAsState()` 简单方便。

`Animatable` 的用法

Kotlin 复制代码

```kotlin
// 用 Animatable(initialValue, typeConverter) 来创建对象
val anim = remember { Animatable(size, Dp.VectorConverter) }
// 如果在 Composable 环境，要带上 remember {}

// 在协程内执行动画流程
LaunchedEffect(big) {
  // snapTo()：立即把值修改为目标值
  anim.snapTo(if (big) 192.dp else 0.dp)
  // animateTo()：以动画形式把目标值渐变为目标值
  anim.animateTo(if (big) 96.dp else 48.dp)
}

...

// 用 Animatable.value 来获取动画属性的实时值
Modifier.size(anim.value)
```

`TwoWayConverter`

用于把动画属性和 Compose 内部的 `AnimationVector1D/2D/3D/4D` 做转化计算用。常用的类型（例如 `Float`、`Dp`、`Size` 等）可以直接使用诸如 `Dp.VectorConverter` 这样的自带的 `TwoWayConverter`。只有特殊类型的属性才需要自定义 `TwoWayConverter`。

`LaunchedEffect()`

- 用于启动协程；
- 和普通启动协程的方法（例如 `lifecycleScope.launch()`）的区别：
  - 和 Compose 做了结合，不会在 Recompose（重组）过程中发生意外重启；
  - 可以填写一个或多个 `keyN` 参数，用于在需要的时候主动重启协程。

`Animatable` 和 `animateXxxAsState()` 的区别

- `Animatable` 可以定制动画流程，`animateXxxAsState()` 只针对状态转移的动画场景；
- `Animatable` 是 `animateXxxAsState()` 的底层实现。

怎么选?

- 状态转移类动画，用 `animateXxxAsState()`；流程定制类动画，用 `Animatable`。

- 或者简单粗暴（但也很准确和实用）地：能用 `animateXxxAsState()` 就用，用不了了再用 `Animatble` 。